
Simulando Qubits com PySCF

Uma Jornada do Zero aos Pontos Quânticos
para a Computação Quântica

Do primeiro `import pyscf` ao qubit de spin em Si:P

Cristiano Alves

Edição de Trabalho — Volume I

28 de julho de 2026

A modelagem computacional de *qubits* baseados em pontos quânticos deixou de ser uma curiosidade acadêmica para se tornar uma competência estratégica. Governos e empresas investem bilhões em computação quântica, e há uma demanda crescente por profissionais capazes de descrever, com rigor atômico, os dispositivos que hão de sustentar essa tecnologia. No entanto, quem tenta ingressar na área costuma esbarrar em dois obstáculos: pacotes de estrutura eletrônica com curvas de aprendizado íngremes e, muitas vezes, com custos de licenciamento proibitivos.

O PySCF — *Python-based Simulations of Chemistry Framework* — derruba essas barreiras. É livre, gratuito e integralmente programável em Python, a linguagem que se tornou a *língua franca* da ciência computacional. Ainda assim, mesmo uma ferramenta amigável pode intimidar quem nunca executou um cálculo de química quântica ou escreveu uma linha de código.

Este livro nasce de uma convicção simples: **é possível partir do zero absoluto e chegar à simulação completa de um *qubit* real**. Nós o faremos com uma abordagem prática, na qual cada conceito é apresentado por meio de exemplos executáveis. Mais do que isso, todo o material converge para um **projeto longitudinal único** — a construção, passo a passo, de um modelo de *qubit* de spin em silício dopado com fósforo (Si:P). Você começará digitando `import pyscf` e terminará extraindo o acoplamento hiperfino, o tensor g , a energia de troca entre *qubits* e estimativas de tempos de descoerência.

Este primeiro volume cobre a **Parte 0 — Preparando o Laboratório Computacional**. O objetivo é claro: garantir que, ao virar a última página do Capítulo 1, você tenha um ambiente Python funcional, domine os elementos da linguagem indispensáveis à química quântica e esteja pronto para instalar e executar o PySCF. Nada será pressuposto; tudo será construído.

Cristiano Alves

Como Usar Este Livro

Este livro foi projetado para ser lido *com o computador ligado*. Cada trecho de código é curto o bastante para ser digitado à mão — e recomendamos que você o digite, pois o aprendizado se consolida pelos dedos, não apenas pelos olhos. Ao longo do texto você encontrará elementos visuais recorrentes:

Nota

Aprofunda um conceito, esclarece uma sutileza ou conecta o assunto a capítulos futuros. Ler é recomendável, mas o fluxo principal não depende disso.

Dica

Truques práticos, atalhos e boas práticas que economizam tempo e evitam frustrações comuns.

Atenção

Erros frequentes e armadilhas. Preste atenção especial: quase todo iniciante tropeça aqui pelo menos uma vez.

Mãos à obra

Exercícios guiados de digitar-e-executar. É onde a teoria vira prática. Não pule estas seções.

Projeto Âncora

Sinaliza um avanço no **Projeto Âncora**, o fio condutor que atravessa todo o livro. Cada etapa acrescenta uma peça ao seu *pipeline* de simulação de *qubits*.

Convenções tipográficas. Nomes de comandos, funções e trechos curtos de código aparecem em fonte monoespaçada. Blocos de código Python são numerados à esquerda e destacados em fundo cinza. Saídas de terminal aparecem em fundo escuro.

Dica

Todo o código deste livro estará disponível em um repositório on-line, em forma de *notebooks* Jupyter prontos para executar. Ainda assim, digite os exemplos ao menos uma vez: a fluência não se baixa, se constrói.

Prefácio	i
Como Usar Este Livro	ii
Preparando o Laboratório Computacional	1
1 Python Essencial para Química Quântica	3
1.1 Por Que Python (e Por Que do Zero)	3
1.2 Variáveis e Tipos de Dados	4
1.2.1 Os tipos fundamentais	4
1.2.2 Aritmética e a unidade atômica de energia	5
1.3 Estruturas de Dados: Listas, Tuplas e Dicionários	5
1.3.1 Listas	5
1.3.2 Tuplas	6
1.3.3 Dicionários	6
1.4 NumPy: O Arsenal Numérico	7
1.4.1 Importando e criando arrays	7
1.4.2 Operações vetorizadas	8
1.4.3 Álgebra linear: o coração da mecânica quântica	8
1.5 Controle de Fluxo: Condicionais e Laços	9
1.5.1 Condicionais	9
1.5.2 O laço <code>for</code>	9
1.6 Funções e Módulos	10
1.7 Ambientes de Desenvolvimento: Jupyter e Google Colab	11
1.7.1 O paradigma do <i>notebook</i>	11
1.7.2 Jupyter Notebook (local)	11
1.7.3 Google Colab (nuvem)	12
1.8 Instalação do PySCF	12
1.8.1 Google Colab (recomendado para começar)	12
1.8.2 Linux e macOS	12
1.8.3 Windows via WSL	13
1.9 Mãos à Obra: Verificando a Instalação	13

Resumo do Capítulo	14
Exercícios	14
2 O Primeiro Cálculo com PySCF	16
2.1 A Anatomia de um Cálculo em PySCF	16
2.2 Definindo uma Molécula: o Objeto <code>Mole</code>	17
2.2.1 A geometria: o argumento <code>atom</code>	17
2.2.2 Inspeccionando o objeto <code>Mole</code>	17
2.3 O Que é uma Base?	18
2.3.1 Famílias de bases	18
2.3.2 O efeito da base na prática	19
2.4 Executando Hartree-Fock: o Objeto <code>SCF</code>	19
2.5 Interpretando a Saída	20
2.5.1 Energias orbitais, ocupações e o <i>gap</i>	21
2.6 Mãos à Obra: do Átomo de Hidrogênio à Molécula H_2	22
Resumo do Capítulo	23
Exercícios	24

Parte

**Preparando o Laboratório
Computacional**

Antes de simular um único elétron, precisamos de um laboratório. No mundo da química quântica computacional, esse laboratório é inteiramente digital: um interpretador **Python**, um punhado de bibliotecas científicas e o **PySCF**. Esta parte inicial tem uma missão bem delimitada — levar você de um computador “em branco” a um ambiente capaz de executar seu primeiro cálculo de estrutura eletrônica.

O Capítulo 1 estabelece a fundação: os elementos da linguagem **Python** que usaremos repetidamente, a biblioteca **NumPy** (o coração numérico de todo o livro), os ambientes de trabalho (Jupyter e Google Colab) e, por fim, a instalação do **PySCF** nos principais sistemas operacionais. O Capítulo 2 executará o primeiro cálculo de verdade. Ao final da Parte 0, o **Projeto Âncora** dará seu primeiro passo com o cálculo do átomo de silício.

Python Essencial para Química Quântica

Todo cálculo que faremos neste livro — do átomo de hidrogênio isolado ao nanocristal de silício com centenas de elétrons — será orquestrado por scripts Python. Python não é apenas a linguagem em que o PySCF foi escrito; é a interface pela qual definiremos moléculas, escolheremos métodos, dispararemos cálculos e, sobretudo, analisaremos resultados.

A boa notícia é que você não precisa se tornar um programador profissional. Você precisa de um subconjunto pequeno, porém sólido, da linguagem: saber guardar valores em variáveis, organizar dados em listas e dicionários, manipular vetores e matrizes com NumPy, repetir tarefas com laços e encapsular lógica em funções. Este capítulo cobre exatamente esse subconjunto — nem mais, nem menos — sempre com exemplos enraizados na química quântica.

Nota

Se você já programa em Python, sintase à vontade para folhear rapidamente as Seções 1.2 a 1.6 e concentrar-se na Seção 1.4 (NumPy) e nas Seções 1.7 e 1.8, sobre ambientes de trabalho e instalação do PySCF.

1.1 Por Que Python (e Por Que do Zero)

Nas últimas duas décadas, Python tornou-se a linguagem dominante da ciência computacional. Três razões explicam essa hegemonia. Primeiro, a **legibilidade**: a sintaxe de Python aproxima-se do pseudocódigo, o que reduz a distância entre a ideia e a implementação. Segundo, o **ecossistema**: bibliotecas como NumPy, SciPy, Matplotlib e o próprio PySCF formam um arsenal maduro e interoperável. Terceiro, a **gratuidade e a portabilidade**: o mesmo script roda em um notebook pessoal, num servidor de alto desempenho ou na nuvem gratuita do Google Colab.

Para nós, químicos quânticos, há um argumento adicional: Python atua como uma *cola* entre componentes de alto desempenho. As rotinas numéricas pesadas do PySCF são executadas em C e Fortran otimizados; Python apenas as comanda. Escrevemos código legível e obtemos desempenho de código compilado — o melhor dos dois mundos.

Nota

Compilada vs. interpretada. Linguagens como C precisam ser *compiladas* — traduzidas para linguagem de máquina — antes de rodar. Python é *interpretada*: cada linha é executada na hora, o que torna o desenvolvimento ágil e interativo, ideal para explorar dados científicos. O custo em velocidade é contornado delegando os cálculos intensos a bibliotecas compiladas, exatamente como o PySCF faz.

1.2 Variáveis e Tipos de Dados

Uma **variável** é um nome que aponta para um valor guardado na memória. Em Python, criar uma variável é tão simples quanto atribuir um valor a um nome com o sinal de igual:

Listing 1.1: Primeiras variáveis: dados de um átomo de hidrogênio.

```

1 # Numero atomico e simbolo do hidrogenio
2 numero_atomico = 1
3 simbolo = "H"
4
5 # Energia do estado fundamental do atomo de H, em hartree
6 energia_fundamental = -0.5
7
8 # Numero de eletrons
9 n_eletrons = 1
10
11 print(simbolo, "tem", n_eletrons, "eletron e energia", energia_fundamental, "Ha")

```

Ao executar esse trecho, você verá:

```
H tem 1 eletron e energia -0.5 Ha
```

Repare em alguns pontos importantes. As linhas iniciadas por # são **comentários**: Python as ignora, mas elas são preciosas para quem lê o código (inclusive você, semanas depois). Não declaramos “tipos”: Python infere automaticamente que `numero_atomico` é um inteiro e `energia_fundamental`, um número de ponto flutuante.

1.2.1 Os tipos fundamentais

Quatro tipos básicos cobrem a quase totalidade das nossas necessidades:

- **Inteiros** (`int`) — contagens exatas: número de elétrons, carga, multiplicidade de spin. Ex.: `n_eletrons = 1`.
- **Ponto flutuante** (`float`) — números reais aproximados: energias, coordenadas, distâncias. Ex.: `energia = -1.174`.
- **Texto** (`str`) — sequências de caracteres: símbolos atômicos, nomes de bases, rótulos. Ex.: `base = "sto-3g"`.
- **Booleanos** (`bool`) — verdadeiro ou falso, usados em decisões. Ex.: `convergiu = True`.

A função embutida `type()` revela o tipo de qualquer valor, e a `print()` mostra resultados na tela:

Listing 1.2: Inspeccionando tipos com `type()`.

```

1 carga = 0                # int
2 distancia_HH = 0.74      # float, em angstrom
3 base = "sto-3g"          # str
4 convergiu = True         # bool
5
6 print(type(carga))       # <class 'int'>
7 print(type(distancia_HH)) # <class 'float'>
8 print(type(base))        # <class 'str'>
9 print(type(convergiu))   # <class 'bool'>

```

1.2.2 Aritmética e a unidade atômica de energia

Python funciona como uma calculadora poderosa. Os operadores básicos são `+`, `-`, `*` (multiplicação), `/` (divisão), `**` (potência) e `%` (resto). Vejamos um cálculo com significado físico: converter uma energia de *hartree* para elétron-volt (1 hartree $\approx$ 27,2114 eV).

Listing 1.3: Conversão de unidades de energia.

```

1 HARTREE_EM_EV = 27.2114      # constante de conversao
2
3 energia_H2_hartree = -1.11676 # energia da molecula H2 (Hartree-Fock/STO-3G)
4 energia_H2_eV = energia_H2_hartree * HARTREE_EM_EV
5
6 print("Energia da H2:", energia_H2_eV, "eV")
7 # Energia da H2: -30.390... eV

```

Nota

Unidades atômicas. A química quântica trabalha em *unidades atômicas* (u.a.), nas quais a massa do elétron, a carga elementar, a constante de Planck reduzida $\hbar$ e a constante de Coulomb valem exatamente 1. A unidade de energia é o *hartree* (Ha) e a de comprimento é o *bohr* ($a_0 \approx 0,529$ Å). Essa escolha elimina constantes das equações e é o padrão interno do PySCF. Guarde as conversões 1 Ha $\approx$ 27,2114 eV e 1 bohr $\approx$ 0,529 Å: elas reaparecerão o tempo todo.

Atenção

A divisão `/` sempre produz um `float` em Python 3, mesmo quando o resultado é “redondo”: `4 / 2` devolve `2.0`, não `2`. Para divisão inteira (descartando a parte fracionária) use `//`. Confundir os dois é uma fonte clássica de *bugs* sutis.

1.3 Estruturas de Dados: Listas, Tuplas e Dicionários

Raramente lidamos com um único número. Uma molécula tem vários átomos; um cálculo produz muitas energias orbitais. Python oferece estruturas para agrupar valores.

1.3.1 Listas

Uma **lista** é uma coleção ordenada e modificável de valores, delimitada por colchetes:

Listing 1.4: Energias orbitais em uma lista.

```

1 # Energias dos orbitais moleculares (em hartree), do mais baixo ao mais alto
2 energias_orbitais = [-0.578, 0.670, 0.965, 1.410]
3
4 print("Numero de orbitais:", len(energias_orbitais)) # 4
5 print("HOMO (ultimo ocupado):", energias_orbitais[0]) # -0.578
6 print("Ultimo da lista:", energias_orbitais[-1]) # 1.410

```

Note duas convenções cruciais de Python. Primeiro, a **indexação começa em zero**: o primeiro elemento é `energias_orbitais[0]`. Segundo, índices negativos contam a partir do fim: `[-1]` é o último elemento. A função `len()` devolve o tamanho da coleção.

Listas são *modificáveis*: podemos acrescentar, remover e alterar elementos.

Listing 1.5: Modificando uma lista.

```

1 distancias = [0.70, 0.74, 0.80] # distancias H-H em angstrom
2 distancias.append(0.90) # adiciona ao final
3 distancias[0] = 0.72 # altera o primeiro elemento
4
5 print(distancias) # [0.72, 0.74, 0.8, 0.9]
6
7 # "Fatiamento" (slicing): sublista do indice 1 (inclusive) ao 3 (exclusive)
8 print(distancias[1:3]) # [0.74, 0.8]

```

Dica

O *fatiamento* `lista[inicio:fim]` inclui o índice *inicio* e *exclui* o *fim*. Uma forma de lembrar: o comprimento da fatia é `fim - inicio`. `distancias[1:3]` tem $3 - 1 = 2$ elementos.

1.3.2 Tuplas

Uma **tupla** é como uma lista, mas *imutável*: uma vez criada, não pode ser alterada. Usamos tuplas para agrupar dados que formam uma unidade lógica e não devem mudar — por exemplo, as coordenadas (x, y, z) de um átomo:

Listing 1.6: Coordenadas atômicas como tuplas.

```

1 # Posicao de um atomo de hidrogenio no espaco (x, y, z) em angstrom
2 posicao_H = (0.0, 0.0, 0.371)
3
4 print("Coordenada z:", posicao_H[2]) # 0.371
5 # posicao_H[2] = 0.5 # ISTO GERARIA ERRO: tuplas sao imutaveis

```

1.3.3 Dicionários

Um **dicionário** associa *chaves* a *valores*, permitindo buscas por nome em vez de por posição. É a estrutura ideal para tabelas de propriedades:

Listing 1.7: Tabela de números atômicos com dicionário.

```

1 # Mapeia simbolo do elemento -> numero atomico
2 numeros_atomicos = {
3     "H": 1,

```

```

4     "C": 6,
5     "N": 7,
6     "O": 8,
7     "Si": 14,
8     "P": 15,
9 }
10
11 print("Numero atomico do fosforo:", numeros_atomicos["P"])    # 15
12 print("Elementos tabelados:", list(numeros_atomicos.keys()))
13
14 # Adicionando uma nova entrada
15 numeros_atomicos["Ga"] = 31
16 print("Galio agora vale:", numeros_atomicos["Ga"])            # 31

```

Nota

Silício (Si, $Z = 14$) e fósforo (P, $Z = 15$) são os protagonistas do nosso Projeto Âncora: construiremos um *qubit* de spin a partir de um átomo de fósforo substituindo um silício numa rede cristalina. Guarde bem esses dois símbolos.

1.4 NumPy: O Arsenal Numérico

Se Python é a linguagem da química quântica computacional, o NumPy é o seu dialeto numérico. A mecânica quântica é, em sua essência, **álgebra linear**: estados são vetores, operadores são matrizes, e energias são autovalores. O NumPy fornece o objeto `ndarray` (*array* N-dimensional) e um vasto conjunto de operações vetoriais rápidas para manipulá-lo. Praticamente tudo o que o PySCF devolve — coeficientes de orbitais, matrizes de densidade, integrais — vem na forma de *arrays* NumPy.

1.4.1 Importando e criando arrays

Bibliotecas em Python são carregadas com a palavra-chave `import`. Por convenção universal, o NumPy é importado com o apelido `np`:

Listing 1.8: Criando arrays com NumPy.

```

1 import numpy as np
2
3 # Um vetor de energias orbitais
4 energias = np.array([-0.578, 0.670, 0.965, 1.410])
5
6 # Uma matriz 2x2 (por exemplo, um Hamiltoniano simples)
7 H = np.array([[0.0, -1.0],
8               [-1.0, 0.0]])
9
10 print("Formato do vetor:", energias.shape)    # (4,)
11 print("Formato da matriz:", H.shape)          # (2, 2)

```

O atributo `.shape` descreve as dimensões do *array*: `(4,)` é um vetor de quatro elementos; `(2, 2)` é uma matriz de duas linhas e duas colunas.

1.4.2 Operações vetorizadas

A grande vantagem do NumPy é a **vetorização**: operações se aplicam a todos os elementos de uma vez, sem laços explícitos — com sintaxe limpa e velocidade de código compilado.

Listing 1.9: Vetorização: convertendo um vetor inteiro de unidades.

```

1 import numpy as np
2
3 energias_hartree = np.array([-0.578, 0.670, 0.965, 1.410])
4
5 # Multiplica CADA elemento por 27.2114, de uma so vez
6 energias_eV = energias_hartree * 27.2114
7
8 print(energias_eV)
9 # [-15.728... 18.231... 26.259... 38.368...]
10
11 # Estatísticas embutidas
12 print("Energia minima:", energias_hartree.min())    # -0.578
13 print("Energia media:", energias_hartree.mean())
14 print("Soma das energias:", energias_hartree.sum())

```

Dica

Sempre que estiver prestes a escrever um laço para percorrer um *array* e fazer contas elemento a elemento, pergunte-se: “existe uma operação NumPy vetorizada para isto?”. Quase sempre existe, e ela será mais curta e muito mais rápida.

1.4.3 Álgebra linear: o coração da mecânica quântica

Chegamos ao ponto em que a álgebra linear encontra a física. Considere o Hamiltoniano de dois níveis

$$\hat{H} = \begin{pmatrix} 0 & -t \\ -t & 0 \end{pmatrix},$$

que descreve, por exemplo, um elétron que pode ocupar dois sítios acoplados por uma amplitude de *tunelamento* t . Os **autovalores** dessa matriz são as energias permitidas do sistema, e os **autovetores**, os estados correspondentes. Com NumPy, obtê-los é imediato:

Listing 1.10: Autovalores e autovetores de um Hamiltoniano 2x2.

```

1 import numpy as np
2
3 t = 1.0
4 H = np.array([[0.0, -t],
5               [-t, 0.0]])
6
7 # Para matrizes simetricas (Hermitianas), use eigh
8 autovalores, autovetores = np.linalg.eigh(H)
9
10 print("Energias (autovalores):", autovalores)    # [-1.  1.]
11 print("Estados (colunas):")
12 print(autovetores)

```


O resultado, `[-1. 1.]`, informa que o sistema tem dois níveis de energia, $-t$ e $+t$. O estado fundamental (energia $-t$) é a combinação simétrica dos dois sítios — exatamente a intuição por trás da ligação química e, mais adiante, do acoplamento entre dois *qubits* doadores. Este pequeno exemplo é um microcosmo de tudo o que faremos: montar um operador como matriz e diagonalizá-lo.

Atenção

Para matrizes simétricas reais (ou Hermitianas complexas) — que é o caso de todo Hamiltoniano físico — use `np.linalg.eigh`, não `np.linalg.eig`. A versão `eigh` explora a simetria, é mais rápida e devolve autovalores reais já ordenados. Usar `eig` num Hamiltoniano pode render autovalores com minúsculas partes imaginárias espúrias.

Nota

Produtos que usaremos sempre. O produto matriz-vetor e matriz-matriz em NumPy é feito com o operador `@` (ou `np.dot`). Assim, `H @ psi` aplica o operador $\hat{H}$ ao estado ψ , e o valor esperado da energia $\langle \psi | \hat{H} | \psi \rangle$ escreve-se `psi @ H @ psi` (para ψ real e normalizado). Guardaremos esse padrão para calcular energias mais adiante.

1.5 Controle de Fluxo: Condicionais e Laços

Scripts científicos precisam tomar decisões e repetir tarefas. Para isso, Python oferece condicionais (`if`) e laços (`for`, `while`).

1.5.1 Condicionais

A estrutura `if/elif/else` executa blocos diferentes conforme uma condição. Um exemplo típico: classificar se um cálculo convergiu.

Listing 1.11: Decisão condicional.

```
1 delta_energia = 3e-9 # variacao de energia no ultimo passo do SCF
2 limiar = 1e-8
3
4 if delta_energia < limiar:
5     print("Convergiu! Podemos confiar no resultado.")
6 else:
7     print("Ainda nao convergiu; aumente o numero de iteracoes.")
```

Atenção

Indentação é sintaxe. Ao contrário de outras linguagens, Python usa o *recuo* (espaços à esquerda) para delimitar blocos — não há chaves. Todas as linhas de um mesmo bloco devem ter o mesmo recuo (o padrão é 4 espaços). Misturar espaços e tabulações, ou recuar de forma inconsistente, gera o temido `IndentationError`. Configure seu editor para inserir 4 espaços ao pressionar Tab.

1.5.2 O laço for

O laço `for` percorre os elementos de uma coleção. É a ferramenta perfeita para processar listas de energias, varreduras de distância e conjuntos de átomos.

Listing 1.12: Percorrendo energias com `for`.

```

1 energias_hartree = [-0.578, 0.670, 0.965, 1.410]
2
3 for i, e in enumerate(energias_hartree):
4     e_eV = e * 27.2114
5     print(f"Orbital {i}: {e:8.3f} Ha = {e_eV:8.2f} eV")

```

Aqui aparecem dois recursos valiosos. A função `enumerate()` devolve, a cada volta, o índice `i` e o valor `e`. E as *f-strings* (`f"..."`) permitem inserir variáveis dentro de um texto com formatação controlada: `{e:8.3f}` imprime o número com 3 casas decimais num campo de largura 8. A saída fica alinhada e legível:

```

Orbital 0:  -0.578 Ha =  -15.73 eV
Orbital 1:   0.670 Ha =   18.23 eV
Orbital 2:   0.965 Ha =   26.26 eV
Orbital 3:   1.410 Ha =   38.37 eV

```

Dica

A função `range(n)` gera a sequência $0, 1, \dots, n-1$ e é ideal para repetir uma tarefa um número fixo de vezes: `for i in range(5):`. Combinada com `range(inicio, fim, passo)`, ela também produz varreduras — por exemplo, uma sequência de distâncias interatômicas.

1.6 Funções e Módulos

À medida que os scripts crescem, repetir código torna-se um risco. As **funções** encapsulam um trecho de lógica sob um nome, permitindo reutilizá-lo com diferentes entradas. Definimos funções com `def`:

Listing 1.13: Uma função para converter energias.

```

1 def hartree_para_eV(energia_hartree):
2     """Converte uma energia de hartree para eletron-volt."""
3     return energia_hartree * 27.2114
4
5 # Reutilizando a funcao com diferentes entradas
6 print(hartree_para_eV(-0.5))      # -13.6057
7 print(hartree_para_eV(-1.1173))  # -30.397...

```

A palavra `return` entrega o resultado a quem chamou a função. O texto entre aspas triplas logo após o `def` é a *docstring*: uma breve descrição do que a função faz, que boas ferramentas exibem como documentação.

Funções podem receber vários argumentos, inclusive com valores-padrão. Vejamos uma que estima, de forma simplificada, a energia de uma ligação a partir de dois níveis acoplados — reaproveitando a ideia da Listagem 1.10:

Listing 1.14: Função com valor-padrão e uso de NumPy.

```

1 import numpy as np
2
3 def energia_ligante(t=1.0):
4     """Energia do estado ligante de um sistema de dois sitios,
5     acoplados por uma amplitude de tunelamento t (em hartree)."""

```

```

6   H = np.array([[0.0, -t],
7                 [-t, 0.0]])
8   autovalores = np.linalg.eigh(H)[0]
9   return autovalores[0] # o menor autovalor: estado fundamental
10
11 print(energia_ligante()) # -1.0 (usa t = 1.0, o padrao)
12 print(energia_ligante(0.5)) # -0.5

```

Nota

Módulos e bibliotecas. Um *módulo* é um arquivo `.py` com funções e variáveis reutilizáveis; uma *biblioteca* (ou pacote) é uma coleção de módulos. Já usamos o NumPy via `import numpy as np`. No próximo capítulo, faremos `from pyscf import gto, scf` para trazer os módulos do PySCF que definem moléculas e executam cálculos. Toda a nossa jornada será uma sucessão de importações e chamadas de função.

1.7 Ambientes de Desenvolvimento: Jupyter e Google Colab

Onde, afinal, digitamos e executamos todo esse código? Há muitos caminhos, mas para a química quântica computacional dois ambientes se destacam pela combinação de interatividade e baixa barreira de entrada: o **Jupyter Notebook** e o **Google Colab**.

1.7.1 O paradigma do *notebook*

Um *notebook* é um documento interativo composto de **células**. Há dois tipos principais: células de *código*, que você executa e cujo resultado aparece logo abaixo, e células de *texto* (em Markdown), para anotações, títulos e equações. Essa mistura de código, resultados e narrativa faz do *notebook* o ambiente ideal para a ciência exploratória: você testa uma ideia, vê o gráfico, escreve uma conclusão, tudo no mesmo lugar. É por isso que todos os materiais deste livro serão distribuídos como *notebooks*.

Dica

Atalhos essenciais do Jupyter/Colab: **Shift+Enter** executa a célula atual e passa para a próxima; **Ctrl+Enter** executa sem avançar. Execute as células *na ordem*, de cima para baixo — uma variável definida numa célula só existe depois que ela foi executada.

1.7.2 Jupyter Notebook (local)

O Jupyter roda no seu próprio computador e é parte da distribuição **Anaconda**, a forma mais simples de instalar Python e as bibliotecas científicas de uma só vez. Após instalar o Anaconda (disponível para Windows, macOS e Linux em <https://www.anaconda.com/download>), abra o *Anaconda Navigator* e clique em “Launch” sob o Jupyter Notebook, ou execute no terminal:

```
jupyter notebook
```

Uma aba abrirá no seu navegador. Trabalhar localmente dá controle total e não depende de conexão à internet — ideal para cálculos mais longos.

1.7.3 Google Colab (nuvem)

O **Google Colab** (<https://colab.research.google.com>) executa *notebooks* Python em servidores do Google, gratuitamente, exigindo apenas uma conta Google e um navegador. **Nada precisa ser instalado no seu computador.** Para quem está começando, é o caminho mais rápido: em segundos você tem um ambiente pronto, e as bibliotecas ausentes podem ser instaladas na hora com um comando.

Dica

No Colab (e no Jupyter), comandos de sistema podem ser executados dentro de uma célula precedendo-os por um ponto de exclamação. Assim, `!pip install pyscf` instala o PySCF diretamente no ambiente de nuvem — exatamente o que faremos a seguir.

Nota

Qual escolher? Recomendamos o **Google Colab** para os primeiros passos e para acompanhar este livro sem atritos, pois elimina a etapa de instalação local. Conforme seus cálculos crescerem — nanocristais com muitos elétrons exigem tempo e memória —, uma instalação local via Anaconda passa a valer a pena. Os *notebooks* do livro funcionam nos dois ambientes.

1.8 Instalação do PySCF

Com um ambiente em mãos, resta instalar o protagonista. O PySCF é distribuído como um pacote Python e instala-se com o gerenciador `pip`. As instruções abaixo cobrem os principais cenários.

Atenção

O PySCF oferece suporte pleno a **Linux** e **macOS**. No **Windows**, a instalação nativa é limitada; a forma recomendada é usar o **WSL** (*Windows Subsystem for Linux*) ou, mais simples ainda, o **Google Colab**. Se você usa Windows e quer evitar complicações, comece pelo Colab.

1.8.1 Google Colab (recomendado para começar)

Abra um *notebook* novo e execute, na primeira célula:

```
!pip install pyscf
```

Aguarde alguns segundos até a mensagem de sucesso. Pronto: o PySCF está disponível naquele *notebook*.

1.8.2 Linux e macOS

Com Python 3.10 ou superior instalado, abra um terminal e execute:

```
pip install pyscf
```

Se você usa Anaconda, é boa prática criar um *ambiente* dedicado ao livro, isolando suas bibliotecas de outros projetos:

```
conda create -n pyscf-livro python=3.11
conda activate pyscf-livro
pip install pyscf numpy scipy matplotlib jupyter
```

1.8.3 Windows via WSL

No Windows 10/11, instale o WSL abrindo o PowerShell como administrador e executando `wsl --install`. Após reiniciar e configurar uma distribuição Linux (Ubuntu, por padrão), abra o terminal do Ubuntu e proceda como na seção anterior:

```
sudo apt update
sudo apt install python3-pip
pip install pyscf
```

Dica

Em qualquer cenário, você pode fixar a versão do PySCF para garantir reprodutibilidade, por exemplo `pip install pyscf==2.6.2`. Ao longo do livro usaremos recursos disponíveis a partir do PySCF 2.3; qualquer versão igual ou superior serve.

1.9 Mãos à Obra: Verificando a Instalação

Chegou a hora de fechar o círculo. Vamos confirmar que Python, NumPy e PySCF estão prontos para uso. Abra um *notebook* (Colab ou Jupyter) e, se ainda não o fez nesta sessão, instale o PySCF.

Mãos à obra

Objetivo: verificar que todo o ambiente funciona, executando o primeiro comando de importação do PySCF e imprimindo as versões instaladas.

Passo 1. Numa célula, instale o PySCF (só é preciso uma vez por sessão no Colab):

```
!pip install pyscf
```

Passo 2. Em outra célula, importe as bibliotecas e imprima suas versões:

```
1 import numpy as np
2 import pyscf
3
4 print("NumPy versao:", np.__version__)
5 print("PySCF versao:", pyscf.__version__)
6 print("Ambiente pronto para a jornada!")
```

Resultado esperado (os números de versão podem variar):

```
NumPy versao: 1.26.4
PySCF versao: 2.6.2
Ambiente pronto para a jornada!
```

Se você viu essas três linhas sem mensagens de erro em vermelho, **parabéns**: seu laboratório computacional está montado. Se algo falhou, consulte a caixa de Atenção a seguir.

Atenção

Problemas comuns. (i) `ModuleNotFoundError: No module named 'pyscf'` — a instalação não foi executada nesta sessão; rode `!pip install pyscf` novamente. (ii) No Colab, se a instalação parecer não “pegar”, reinicie o ambiente pelo menu *Runtime* → *Restart runtime* e execute as células na ordem. (iii) No Windows nativo, erros de compilação indicam que você deveria estar usando WSL ou Colab.

Projeto Âncora**Etapla 0 — Nasce o *notebook* mestre.**

O Projeto Âncora deste livro é a construção completa de um *qubit* de spin em silício dopado com fósforo (Si:P). Cada capítulo acrescentará uma peça. A Etapa 0, que você acaba de cumprir, é a fundação: um ambiente funcional.

Sua tarefa. Crie um *notebook* chamado `projeto_ancora_SiP.ipynb`. Na primeira célula de texto (Markdown), escreva um título e uma frase declarando o objetivo do projeto. Na primeira célula de código, cole o bloco de verificação do Mãos à Obra e execute-o. Guarde esse *notebook*: ele crescerá conosco até conter, ao final do livro, todo o *pipeline* de simulação — da geometria do nanocristal ao tempo de descoerência T_2^* .

No próximo capítulo, esse mesmo *notebook* receberá seu primeiro cálculo de química quântica de verdade: a energia do átomo de silício.

Resumo do Capítulo

- Python é a linguagem que orquestra a química quântica computacional: legível, gratuita e cercada de um ecossistema científico maduro.
- **Variáveis** guardam valores de quatro tipos essenciais: inteiros, *floats*, textos e booleanos. Trabalhamos em unidades atômicas (hartree, bohr).
- **Listas, tuplas e dicionários** organizam conjuntos de dados; a indexação começa em zero.
- O NumPy traz *arrays* e álgebra linear vetorizada. Montar um Hamiltoniano como matriz e diagonalizá-lo com `np.linalg.eigh` é o gesto fundamental que repetiremos por todo o livro.
- **Condicionais, laços e funções** dão estrutura e reaproveitamento aos scripts; a indentação define blocos.
- **Jupyter** (local, via Anaconda) e **Google Colab** (nuvem, sem instalação) são os ambientes de trabalho recomendados.
- O PySCF instala-se com `pip install pyscf`; no Windows, prefira WSL ou Colab.
- O **Projeto Âncora** teve início: você tem um *notebook* mestre com o ambiente verificado.

Exercícios

- 1.1 Conversões.** Crie uma variável com a energia $-2,903$ Ha (aproximadamente a energia do átomo de hélio). Converta-a para eV usando o fator 27,2114 e imprima o resultado com uma *f-string*, exibindo 4 casas decimais.

- 1.2 Dicionário de massas.** Monte um dicionário associando os símbolos "H", "Si" e "P" às suas massas atômicas aproximadas (1,008; 28,09; 30,97). Em seguida, imprima a massa do fósforo e adicione uma entrada para o carbono ("C": 12,011).
- 1.3 Varredura com `for`.** Usando `range`, crie uma lista de cinco distâncias interatômicas de 0,70 a 0,90 Å em passos de 0,05. Percorra-a com um laço `for` e imprima cada distância convertida para bohr (divida por 0,529).
- 1.4 Diagonalização.** Adapte a Listagem 1.10 para o Hamiltoniano $\hat{H} = \begin{pmatrix} \varepsilon & -t \\ -t & \varepsilon \end{pmatrix}$ com $\varepsilon = 0,2$ e $t = 1,0$. Imprima os autovalores e verifique, no papel, que eles valem $\varepsilon \pm t$.
- 1.5 Função reutilizável.** Escreva uma função `gap_HOMO_LUMO(energias)` que receba uma lista de energias orbitais *ordenada* e o índice do HOMO, e devolva o *gap* (energia do LUMO menos a do HOMO). Teste-a com a lista da Listagem 1.4, supondo o HOMO no índice 0.
- 1.6 Projeto Âncora.** No seu *notebook* `projeto_ancora_SiP.ipynb`, acrescente uma célula que imprima, além das versões de NumPy e PySCF, também a versão do Python em execução (dica: `import sys; print(sys.version)`). Anote numa célula de texto qual ambiente você escolheu (Colab ou Jupyter) e por quê.

O Primeiro Cálculo com PySCF

No capítulo anterior montamos o laboratório: temos Python, NumPy e o PySCF instalados e verificados. Agora vamos usá-los para o que realmente importa — resolver a equação de Schrödinger eletrônica de um sistema real e obter sua energia. Ao final deste capítulo, você terá calculado, do zero, a energia do átomo de hidrogênio, da molécula H_2 e — como primeiro passo concreto do Projeto Âncora — do átomo de silício.

Não se preocupe se os fundamentos teóricos do método Hartree-Fock ainda parecerem nebulosos: o Capítulo 4 os desenvolverá em detalhe. Aqui, o objetivo é outro e igualmente essencial — **ganhar fluência no fluxo de trabalho do PySCF**: definir um sistema, escolher um método, executar o cálculo e interpretar a saída. Esse quarteto se repetirá, praticamente inalterado, em todos os cálculos do livro.

2.1 A Anatomia de um Cálculo em PySCF

Quase todo cálculo de estrutura eletrônica no PySCF segue três passos:

1. **Definir o sistema** — que átomos, em que posições, com que carga e spin, e em qual conjunto de funções de base. Isso é encapsulado no objeto `Mol` (de *molecule*).
2. **Escolher o método** — Hartree-Fock, DFT, pós-Hartree-Fock. Aqui usaremos o objeto `SCF` na sua forma Hartree-Fock.
3. **Executar e analisar** — disparar o cálculo com `kernel()` e ler os resultados: energia total, energias orbitais, convergência.

Em código, esse esqueleto cabe em cinco linhas:

Listing 2.1: O esqueleto de qualquer cálculo PySCF.

```
1 from pyscf import gto, scf          # 1. importa os modulos
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', # 2. define o sistema
4         basis='sto-3g')
5
6 mf = scf.RHF(mol)                   # 3. escolhe o metodo (Hartree-Fock)
7 energia = mf.kernel()               # 4. executa e obtem a energia total
```


O módulo `gto` (*Gaussian-type orbitals*) cuida da definição do sistema; o módulo `scf` (*self-consistent field*) cuida dos métodos de campo autoconsistente, dos quais o Hartree-Fock é o mais fundamental. Vamos dissecar cada passo.

2.2 Definindo uma Molécula: o Objeto Mole

Tudo começa pela descrição do sistema físico. A função `gto.M()` constrói um objeto `Mole` que reúne toda a informação: geometria, base, carga e spin. Seus argumentos mais importantes são:

Listing 2.2: Os argumentos essenciais de `gto.M`.

```
1 from pyscf import gto
2
3 mol = gto.M(
4     atom='''H 0.0 0.0 0.0
5           H 0.0 0.0 0.74''', # geometria: simbolo x y z (por linha)
6     basis='sto-3g',          # conjunto de funcoes de base
7     charge=0,                # carga total do sistema
8     spin=0,                  # numero de eletrons desemparelhados (2S)
9     unit='Angstrom',         # unidade das coordenadas (padrao)
10    verbose=4,               # nivel de detalhe da saida (0 a 9)
11 )
```

2.2.1 A geometria: o argumento `atom`

A geometria é fornecida como um texto em que cada linha traz um símbolo atômico seguido de suas coordenadas cartesianas x , y , z . Átomos podem ser separados por quebras de linha (como acima) ou por ponto e vírgula na mesma linha (`'H 0 0 0; H 0 0 0.74'`). As coordenadas estão em $\text{\AA}$ por padrão; para usar bohr, defina `unit='Bohr'`.

Atenção

O argumento `spin` no PySCF **não** é o spin total S nem a multiplicidade $2S + 1$: é o *número de elétrons desemparelhados*, $n_\alpha - n_\beta = 2S$. Assim, um sistema de camada fechada (como a H_2 no estado fundamental) tem `spin=0`; o átomo de hidrogênio, com um elétron solitário, tem `spin=1`; o átomo de silício, com dois elétrons $3p$ desemparelhados, tem `spin=2`. Errar esse número é a causa mais comum de falhas em cálculos de átomos e radicais.

2.2.2 Inspeccionando o objeto `Mole`

Uma vez construído, o objeto `mol` sabe muito sobre o sistema. Vale a pena interrogá-lo antes de calcular — é uma checagem barata que evita horas de frustração com um sistema mal definido:

Listing 2.3: Interrogando o objeto `Mole`.

```
1 from pyscf import gto
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='sto-3g')
4
5 print("Numero de eletrons:", mol.nelectron) # 2
6 print("Numero de atomos:", mol.natm)        # 2
```

```

7 print("Numero de funcoes de base (AOs):", mol.nao) # 2
8 print("Carga:", mol.charge, " Spin (2S):", mol.spin) # 0 0

```

```

Numero de eletrons: 2
Numero de atomos: 2
Numero de funcoes de base (AOs): 2
Carga: 0 Spin (2S): 0

```

O atributo `mol.nao` (*number of atomic orbitals*) informa quantas funções de base descrevem o sistema — um número que determina diretamente o custo do cálculo e a qualidade do resultado, como veremos a seguir.

2.3 O Que é uma Base?

Ao resolver a equação de Schrödinger num computador, não podemos representar os orbitais como funções contínuas arbitrárias. Em vez disso, escrevemos cada orbital molecular como uma **combinação linear** de funções conhecidas e fixas — as **funções de base**. O conjunto dessas funções é o *conjunto de base* (*basis set*).

Formalmente, um orbital molecular ψ_i é expandido como

$$\psi_i(\mathbf{r}) = \sum_{\mu=1}^N C_{\mu i} \phi_{\mu}(\mathbf{r}),$$

onde ϕ_{μ} são as funções de base (fixas) e $C_{\mu i}$ são os coeficientes que o cálculo determina. Quanto *maior* e mais flexível o conjunto de base (maior N), melhor a descrição — e maior o custo computacional. Encontrar o equilíbrio entre precisão e custo é uma arte central da química quântica.

Nota

Por que gaussianas? As funções de base do PySCF são *orbitais do tipo gaussiano* (GTOs), da forma $e^{-\alpha r^2}$. Elas não descrevem um átomo tão fielmente quanto as funções exponenciais $e^{-\alpha r}$ (do átomo de hidrogênio real), mas têm uma propriedade matemática preciosa: o produto de duas gaussianas centradas em pontos diferentes é outra gaussiana. Isso torna as integrais de dois elétrons — o gargalo do cálculo — tratáveis analiticamente. Para recuperar o comportamento correto, cada função de base é, na verdade, uma soma fixa (*contração*) de várias gaussianas.

2.3.1 Famílias de bases

Você encontrará algumas famílias de nomes recorrentes:

- **STO-3G** — uma base *mínima*: cada orbital atômico é aproximado por 3 gaussianas contraídas. É pequena, rápida e ótima para aprender e depurar, mas pouco precisa. Usaremos-la como “base de rascunho”.
- **Pople (6-31G, 6-311G**)** — bases de porte médio, clássicas na literatura.
- **correlação consistente** (cc-pVDZ, cc-pVTZ, cc-pVQZ) — bases sistemáticas, projetadas para convergir suave e previsivelmente rumo ao limite. “D”, “T”, “Q” indicam qualidade *double*, *triple*, *quadruple-zeta*.

- **def2** (def2-SVP, def2-TZVP) — família moderna e equilibrada de Karlsruhe, excelente para toda a tabela periódica — útil para os elementos mais pesados dos nossos nanocristais.

2.3.2 O efeito da base na prática

Nada ensina melhor do que os números. A Tabela 2.1 mostra a energia Hartree-Fock da molécula H_2 (à distância de 0,74 Å) calculada com bases crescentes. Observe duas tendências: o número de funções de base (**nao**) cresce, e a energia *desce* monotonamente, aproximando-se do limite Hartree-Fock ($\approx -1,1336$ Ha).

Tabela 2.1: Energia HF da H_2 ($R = 0,74$ Å) em função da base.

Base	Funções de base (nao)	Energia (Ha)
STO-3G	2	-1,116759
cc-pVDZ	10	-1,128700
cc-pVTZ	28	-1,132968
<i>limite Hartree-Fock</i>		$\approx -1,1336$

Dica

Uma energia mais baixa é sempre melhor? No método Hartree-Fock (e em qualquer método *variacional*), **sim**: o *princípio variacional* garante que a energia calculada é sempre um limite *superior* à energia verdadeira. Logo, entre duas bases, a que fornece a energia mais baixa é a mais próxima da realidade. Guarde esse princípio: ele é a bússola que orienta a escolha de bases e métodos.

2.4 Executando Hartree-Fock: o Objeto SCF

Com o sistema definido, escolhemos o método. O **Hartree-Fock** (HF) é o ponto de partida de quase toda a química quântica. Sua ideia central é uma aproximação de *campo médio*: em vez de tratar a repulsão instantânea entre cada par de elétrons — um problema intratável —, supõe-se que cada elétron se move no campo *médio* gerado por todos os outros. Como esse campo depende dos próprios orbitais que queremos encontrar, o problema é resolvido de forma **iterativa e autoconsistente**: daí o nome *Self-Consistent Field* (SCF). O Capítulo 4 detalhará o formalismo; por ora, basta saber que o PySCF faz todo o trabalho pesado.

O PySCF oferece três “sabores” de Hartree-Fock, e escolher o certo depende do spin do sistema:

- **scf.RHF** — *Restricted HF*: para sistemas de camada fechada (**spin=0**), em que todos os elétrons estão emparelhados. É o caso da H_2 .
- **scf.ROHF** — *Restricted Open-shell HF*: para sistemas com elétrons desemparelhados que se deseja tratar de forma restrita. É a escolha natural para átomos como H e Si.
- **scf.UHF** — *Unrestricted HF*: permite que elétrons de spin α e β ocupem orbitais espaciais diferentes. Essencial para radicais e para os cálculos de acoplamento de troca da Parte III.

Criado o objeto de método, o cálculo é disparado com `kernel()`, que devolve a energia total em hartree e a armazena também no atributo `mf.e_tot`:

Listing 2.4: Um cálculo Hartree-Fock completo da H₂.

```

1 from pyscf import gto, scf
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='sto-3g')
4
5 mf = scf.RHF(mol)
6 energia = mf.kernel()
7
8 print("Energia total: %.8f Ha" % energia)
9 print("Confirmando via atributo: %.8f Ha" % mf.e_tot)

```

```

converged SCF energy = -1.11675930739643
Energia total: -1.11675931 Ha
Confirmando via atributo: -1.11675931 Ha

```

A linha `converged SCF energy = ...` é impressa pelo próprio PySCF: ela anuncia que o ciclo autoconsistente convergiu e informa a energia final. Nossa molécula H₂ tem energia Hartree-Fock de $-1,11675931$ Ha.

2.5 Interpretando a Saída

Obter um número é fácil; *confiar* nele exige ler a saída com olhos críticos. Aumentando o nível de detalhe (`verbose=4`), o PySCF revela o andamento do ciclo SCF. Para tornar a leitura instrutiva, usemos a molécula de água, cujo SCF leva alguns ciclos para convergir (a H₂, minúscula, converge quase de imediato):

Listing 2.5: Acompanhando a convergência do SCF.

```

1 from pyscf import gto, scf
2
3 mol = gto.M(atom='''O 0.000 0.000 0.117
4               H 0.000 0.757 -0.469
5               H 0.000 -0.757 -0.469''',
6             basis='sto-3g', verbose=4)
7
8 mf = scf.RHF(mol)
9 mf.kernel()

```

Entre as muitas linhas da saída, as mais importantes são o registro dos ciclos:

```

cycle= 1 E= -74.9128050859 delta_E= -0.0755 |g|= 0.37 |ddm|= 1.69
cycle= 2 E= -74.9624186902 delta_E= -0.0496 |g|= 0.0424 |ddm|= 0.56
cycle= 3 E= -74.9629204358 delta_E= -0.000502 |g|= 0.00832 |ddm|= 0.0432
cycle= 4 E= -74.9629466547 delta_E= -2.62e-05 |g|= 6.37e-05 |ddm|= 0.015
cycle= 5 E= -74.9629466564 delta_E= -1.67e-09 |g|= 1.46e-05 |ddm|= 0.0001
cycle= 6 E= -74.9629466565 delta_E= -1.58e-10 |g|= 3.45e-06 |ddm|= 4.2e-05
converged SCF energy = -74.962946656387

```

Cada linha é uma iteração do campo autoconsistente. Três colunas contam a história da convergência:

- E — a energia total naquele ciclo. Note-a *descendo* monotonamente e estabilizando.

- `delta_E` — a variação de energia em relação ao ciclo anterior. É o critério principal: quando cai abaixo de $\sim 10^{-9}$ Ha, o cálculo convergiu.
- `|g|` — a norma do gradiente (o quanto os orbitais ainda “querem” mudar). Também deve tender a zero.

Atenção

Convergiu $\neq$ correto. A mensagem `converged SCF energy` garante apenas que o algoritmo encontrou uma solução autoconsistente — não que ela seja a solução fisicamente desejada nem que a base seja adequada. E o pior cenário é o oposto: se aparecer `SCF not converged`, o número reportado **não tem valor** e deve ser descartado. Sempre confira se o cálculo convergiu antes de usar qualquer resultado.

2.5.1 Energias orbitais, ocupações e o *gap*

Além da energia total, o objeto de método guarda os **orbitais moleculares** resultantes. Dois atributos são especialmente úteis: `mf.mo_energy` (as energias dos orbitais, em ordem crescente) e `mf.mo_occ` (quantos elétrons ocupam cada orbital). Vejamos para a água:

Listing 2.6: Lendo orbitais, ocupações e o *gap* HOMO–LUMO.

```

1 import numpy as np
2
3 print("Energias orbitais (Ha):", np.round(mf.mo_energy, 4))
4 print("Ocupacoes:", mf.mo_occ)
5
6 # HOMO = ultimo orbital ocupado; LUMO = primeiro vazio
7 n_ocupados = int(mf.mo_occ.sum() // 2)      # 5 orbitais duplamente ocupados
8 homo = mf.mo_energy[n_ocupados - 1]
9 lumo = mf.mo_energy[n_ocupados]
10 gap = lumo - homo
11
12 print("HOMO: %.4f Ha   LUMO: %.4f Ha" % (homo, lumo))
13 print("Gap HOMO-LUMO: %.4f Ha = %.2f eV" % (gap, gap * 27.2114))

```

```

Energias orbitais (Ha): [-20.2418 -1.2684 -0.6179 -0.453 -0.3912 0.6056 0.7422]
Ocupacoes: [2. 2. 2. 2. 2. 0. 0.]
HOMO: -0.3912 Ha   LUMO: 0.6056 Ha
Gap HOMO-LUMO: 0.9968 Ha = 27.12 eV

```

A leitura é rica. A água tem 10 elétrons, distribuídos em 5 orbitais duplamente ocupados (ocupação 2.), enquanto os 2 orbitais restantes estão vazios (0.). O primeiro orbital, com energia profundíssima ($-20,24$ Ha), é essencialmente o caroço $1s$ do oxigênio. O HOMO (*Highest Occupied Molecular Orbital*) e o LUMO (*Lowest Unoccupied*) delimitam o *gap* — a energia mínima para excitar um elétron. Esse *gap* será uma das grandezas centrais do Projeto Âncora: no nanocristal de silício, é ele que traduz o confinamento quântico.

Dica

Quer todos os resultados de uma vez? O método `mf.analyze()` imprime um relatório completo — energias orbitais, ocupações, cargas atômicas e momentos de dipolo — num único comando. É uma forma rápida de “radiografar” um cálculo recém-concluído.

2.6 Mãos à Obra: do Átomo de Hidrogênio à Molécula H₂

Hora de calcular por conta própria. Faremos dois cálculos clássicos e compararemos os resultados com valores conhecidos — o hábito que separa o usuário crítico do apertador de botões.

Mãos à obra

Objetivo: calcular a energia do átomo de H e da molécula H₂, e observar como a energia do átomo de H se aproxima do valor exato (−0,5 Ha) à medida que a base melhora.

Parte A — O átomo de hidrogênio. Com um único elétron, o átomo de H é de camada aberta: use `spin=1` e `ROHF`.

```
1 from pyscf import gto, scf
2
3 for base in ['sto-3g', 'cc-pvdz', 'cc-pvtz']:
4     mol = gto.M(atom='H 0 0 0', basis=base, spin=1)
5     energia = scf.ROHF(mol).kernel()
6     print("%-8s -> %.6f Ha" % (base, energia))
```

```
sto-3g    -> -0.466582 Ha
cc-pvdz   -> -0.499278 Ha
cc-pvtz   -> -0.499810 Ha
```

Observe a energia subindo... digo, *descendo* rumo a −0,5 Ha, o valor exato. Com a base mínima STO-3G o erro é grande (~ 7%); com cc-pVTZ, cai para 0,04%. A química quântica computacional é, em boa medida, a arte de controlar esse tipo de erro.

Parte B — A molécula H₂. Camada fechada: `spin=0` e `RHF`.

```
1 from pyscf import gto, scf
2
3 mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='cc-pvdz')
4 mf = scf.RHF(mol)
5 energia = mf.kernel()
6
7 print("Energia da H2: %.6f Ha" % energia)
8 print("HOMO: %.4f Ha" % mf.mo_energy[0])
```

```
converged SCF energy = -1.12870045...
Energia da H2: -1.128700 Ha
HOMO: -0.5924 Ha
```

Compare com a energia dos dois átomos de H isolados ($2 \times -0,499 = -0,998$ Ha na mesma base): a H₂ é bem *mais* estável, e a diferença é, essencialmente, a energia da ligação química.

Atenção

Camada fechada vs. aberta. Se você tentar rodar `scf.RHF` num átomo de H com `spin=1`, o PySCF reclamará — RHF pressupõe todos os elétrons emparelhados. E se esquecer o

`spin=1` (deixando o padrão `spin=0`) num sistema de número ímpar de elétrons, receberá um erro de consistência. Regra prática: **conte os elétrons desemparelhados e informe-os em `spin`.**

Projeto Âncora

Etapa 0 (conclusão) — O primeiro cálculo do átomo de silício.

O átomo de silício é o alicerce do nosso nanocristal. Antes de agrupá-lo às centenas, precisamos saber descrevê-lo isoladamente. Silício tem 14 elétrons e configuração $[Ne] 3s^2 3p^2$; os dois elétrons $3p$ ocupam orbitais distintos (estado fundamental 3P), logo há **dois elétrons desemparelhados**: `spin=2`.

Sua tarefa. No *notebook* `projeto_ancora_SiP.ipynb`, acrescente uma célula com o cálculo abaixo e execute-o.

```
1 from pyscf import gto, scf
2
3 # Atomo de silicio isolado: 14 eletrons, estado fundamental 3P (spin=2)
4 mol_si = gto.M(atom='Si 0 0 0', basis='sto-3g', spin=2)
5 print("Eletrons:", mol_si.nelectron, " AOs:", mol_si.nao)
6
7 mf_si = scf.ROHF(mol_si)
8 e_si = mf_si.kernel()
9 print("Energia do atomo de Si (STO-3G): %.6f Ha" % e_si)
```

```
Eletrons: 14 AOs: 9
converged SCF energy = -285.466211172452
Energia do atomo de Si (STO-3G): -285.466211 Ha
```

Repetindo com a base `def2-svp` (18 funções de base, mais precisa), a energia desce para $-288,749819$ Ha — coerente com o princípio variacional. Anote os dois valores no *notebook*.

Marco atingido. Com este cálculo, a Parte 0 está completa: você tem um ambiente funcional, domina o fluxo de trabalho do PySCF e executou seu primeiro cálculo do átomo que protagonizará todo o livro. Na Parte I, deixaremos de tratar um átomo isolado e começaremos a *construir o nanocristal*, agrupando dezenas de átomos de silício e inserindo o doador de fósforo.

Resumo do Capítulo

- Todo cálculo PySCF segue três passos: **definir o sistema** (`gto.M` → objeto `Mole`), **escolher o método** (`scf.RHF/ROHF/UHF`) e **executar** (`kernel()`).
- No `Mole`, a geometria vai no argumento `atom` (símbolo + $x y z$, em $\text{\AA}$ por padrão); `charge` é a carga e `spin` é o número de elétrons *desemparelhados* ($2S$), não a multiplicidade.
- Uma **base** expande os orbitais em funções gaussianas fixas. Bases maiores (`STO-3G` → `cc-pVDZ` → `cc-pVTZ`) dão energias menores e mais precisas, ao custo de mais funções (`nao`).
- Pelo **princípio variacional**, a energia HF é sempre um limite superior à energia verdadeira:

menor é melhor.

- Escolha RHF para camada fechada, ROHF/UHF para camada aberta (átomos, radicais).
- Sempre verifique a **convergência** (`converged SCF energy`) antes de confiar num resultado; leia `mo_energy` e `mo_occ` para obter HOMO, LUMO e o *gap*.
- **Projeto Âncora:** calculamos o átomo de silício isolado ($-285,466$ Ha em STO-3G), encerrando a Parte 0.

Exercícios

- 2.1 Camada fechada.** Calcule a energia Hartree-Fock do átomo de hélio (`atom='He 0 0 0', spin=0`) com RHF nas bases STO-3G e cc-pVTZ. Compare com o valor de referência ($\approx -2,86$ Ha no limite HF) e comente qual base chega mais perto.
- 2.2 O papel do spin.** Tente calcular o átomo de hidrogênio com `scf.RHF` e `spin=0`. Leia a mensagem de erro e explique, em uma frase, por que ela ocorre. Em seguida, corrija usando `spin=1` e ROHF.
- 2.3 Curva de dissociação (esboço).** Usando um laço `for` sobre uma lista de distâncias H–H (por exemplo, 0,5 a 2,0 Å em passos de 0,25), calcule e imprima a energia RHF/STO-3G da H_2 em cada distância. Em qual distância a energia é mínima?
- 2.4 Inspeccionando orbitais.** Para a molécula de água da Listagem 2.5, imprima `mf.mo_occ` e confirme que há 5 orbitais ocupados. Calcule o *gap* HOMO–LUMO em eV.
- 2.5 Efeito da base no gap.** Calcule o *gap* HOMO–LUMO da H_2 nas bases STO-3G e cc-pVDZ. O *gap* aumenta ou diminui ao melhorar a base? Proponha uma explicação.
- 2.6 Projeto Âncora.** No seu *notebook*, calcule o átomo de silício (`spin=2, ROHF`) nas bases `sto-3g` e `def2-svp`. Registre numa célula de texto: (a) a energia em cada base; (b) o número de funções de base (`nao`); (c) por que a base maior fornece energia mais baixa.